

데이터 플랫폼 on kubernetes

2026.04 -

Description: 데이터 플랫폼 운영 환경을 Kubernetes—EKS 환경으로 옮기는 작업을 주도했다. 플랫폼 컴포넌트의 배포 구성을 Helm Chart 로 관리하고, ArgoCD 기반 GitOps 패턴을 적용할 수 있도록 배포 구조를 정리했다.

• Problem

- ASG(EC2) 기반의 서비스를 제공하며 리소스 스케일아웃 에 대한 관리 복잡도가 증가함
- Managed 서비스 이용 및 비 컨테이너 환경으로 구성된 플랫폼으로 환경별 차이를 추적하기 어렵고 데이터팀 CICD 작업의 비효율이 증가함
- 플랫폼 운영 컴포넌트가 늘어날수록 배포 방식, Secret 관리, 환경별 설정을 같은 기준으로 유지하기 어려움 (코드로 관리할 수 없음)

• Action

- EKS 클러스터 신규 구성
- Kubernetes/EKS 운영을 전제로 Helm Chart 와 ArgoCD Application/ApplicationSet 구조
- ArgoCD, AWS Load Balancer Controller, Secrets Store CSI Driver 등 플랫폼 운영 컴포넌트의 Chart 및 환경별 values 정리
- Observability—Datadog(datadog agent operator)

• Result

- ASG(EC2) 기반으로 운영하던 데이터 플랫폼 컴포넌트를 EKS 기반으로 배포할 수 있는 기본 구조를 마련
- Helm values 와 ArgoCD 매니페스트를 기준으로 환경별 설정 차이, 배포 구성, Secret 참조 방식을 코드에서 함께 확인할 수 있게 정리
- 신규 플랫폼 컴포넌트 추가 시 공통 Chart 구조와 GitOps 배포 흐름을 재사용할 수 있도록 운영 기준을 표준화

• Impact

- 리소스 확장과 배포 변경을 개별 서버 단위로 처리하던 방식에서 Kubernetes 선언형 구성 기반의 운영 방식으로 전환
- 환경별 구성 차이와 Secret 관리 기준이 명확해져 데이터팀의 배포 검토 및 CICD 운영 부담을 낮춤
- 이후 Flink, Kafka Connect 등 데이터 플랫폼 컴포넌트에도 같은 배포/운영 패턴을 적용할 수 있는 기반 확보

• Self-Reflection

- 애플리케이션을 이미지로 만드는 일에 그치지 않고, 배포 방식과 운영 설정 관리까지 함께 바꾸는 작업임을 체감
- 플랫폼 컴포넌트—Helm Chart, ArgoCD, Secret 관리가 맞물려야 안정적인 플랫폼 운영이 가능하다는 점을 배움
- 데이터 엔지니어링 업무에서도 Kubernetes 이해가 중요해지고 있음을 실무적으로 경험

• Architecture Decision

- **Kubernetes:** ASG 에 사용한 스펙을 관리하지 않고 선언형 방식의 오케스트레이션이 필요했음. 작업 변경—특히 resource 이 코드로 관리할 수 있었음. 오버엔지니어링으로 느낄 수 있지만 기존 SaaS 서비스를 내제화 하여 플랫폼 컴포넌트에 대한 운영 경험을 쌓을 수 있음(Not Blackbox)
- **Helm/ArgoCD:** 플랫폼 컴포넌트가 늘어날수록 manifest 를 직접 복사해 관리하는 방식으로는 환경별 설정 차이와 변경 이력을 일관되게 추적하기 어렵다고 판단함. Helm 으로 공통 리소스 구조와 환경별 values 를 분리하고, ArgoCD 로 Git 을 기준으로 한 선언형 배포 흐름을 구성함. 별도 배포 도구를 직접 만들기보다 Kubernetes 생태계에서 널리 쓰이는 조합(de facto)을 채택해 운영 부담과 팀 내 학습 비용을 줄이는 방향을 선택함.

Description: Airflow orchestration 코드와 분리해 Managed Flink 애플리케이션을 관리할 수 있는 구조를 만들었다. Kafka CDC 이벤트를 Flink Table API로 읽어 Iceberg 테이블에 적재하는 Sink 애플리케이션—changelog/materialized 을 개발했다. 매니지드 서비스를 사용할 수 밖에 없는 환경에 리소스를 코드로—Terraform 정리했고 애플리케이션 코드 변경으로 발생하는 CICD 파이프라인도 마련했다.

• Problem

- 시간단위 배치보다 더 짧은 주기—Interval 의 데이터 제공의 수요가 증가함
- Kafka Sink Connector 로 간단한 데이터 파이프라인을 제공할 수 있었지만 비즈니스로직을 담은 수요도 함께 증가

• Action

- multi-module 기반 Flink 애플리케이션 구성 및 JAR 빌드 환경
- Table API 기반—Everything is SQL 구현: CDC 메시지를 읽어 Iceberg 테이블로 적재
- changelog/materialized 적재 모드 지원으로 원본 이벤트 누적과 최신 상태 테이블 구성을 분리
- 복잡한 비즈니스 로직을 담을 수 있는 애플리케이션 boilerplate 구성

• Result

- Kafka CDC 이벤트를 Flink Table API 로 처리해 Iceberg 테이블에 적재하는 재사용 가능한 Sink 애플리케이션을 구현
- changelog 와 materialized 적재 모드를 분리해 원본 이벤트 보관과 최신 상태 테이블 구성을 같은 애플리케이션 구조에서 지원
- multi-module 구조와 JAR 빌드 환경을 정리해 Airflow DAG 와 분리된 Flink 애플리케이션 개발/배포 단위를 마련
- Managed Flink 실행 설정과 테이블 naming rule 을 코드와 설정으로 관리해 환경별 운영 차이를 줄임

• Impact

- 시간 단위 배치로 제공하던 데이터를 더 짧은 주기로 적재할 수 있는 스트리밍 기반 처리 경로를 확보—a.k.a 준실시간
- Kafka Sink Connector 만으로 처리하기 어려운 비즈니스 로직을 Flink 애플리케이션 레이어에서 다룰 수 있게 됨
- Kafka CDC, Flink, Iceberg 로 이어지는 Lakehouse 적재 패턴을 구축해 이후 실시간성 데이터 파이프라인 확장의 기준을 마련
- Airflow orchestration 중심의 운영 범위를 stream processing, table format, catalog 관리까지 확장

• Self-Reflection

- Flink 애플리케이션은 DAG 스크립트와 달리 빌드 산출물, 런타임 설정, 테이블 스키마를 함께 고려해야 한다는 점을 체감
- CDC 이벤트를 Iceberg 테이블로 적재하며 append-only 로그와 materialized view 성격의 테이블 설계 차이를 더 명확히 이해
- 데이터 파이프라인 운영 범위가 orchestration 을 넘어 stream processing, table format, catalog 관리까지 확장된다는 점을 경험

• Architecture Decision

- **Apache Flink:** 단순 Sink 처리만으로는 CDC 이벤트에 필요한 비즈니스 로직을 충분히 담기 어렵다고 판단. ksqldb 도 선택지로 볼 수 있었지만, 복잡한 변환 로직을 코드로 표현하고 테스트 가능한 애플리케이션 단위로 관리하기에는 Flink 가 더 적합하다고 판단. 제이슨 크랩스에 따르면 더 이상 Confluent 핵심 리소스로 보지 않는다고 선언함. Table API 를 활용해 SQL 기반의 처리 흐름은 유지하면서도, 필요할 때 Java/Scala/Python 코드로 로직을 확장할 수 있는 구조를 선택함. (ksqldb—Only SQL)

- **Managed Service:** 초기에는 Kubernetes Operator 기반 배포도 고려했지만, Flink 클러스터 운영까지 팀이 직접 감당하면 애플리케이션 개발보다 운영 복잡도가 먼저 커질 수 있다고 판단함. 런타임 운영 부담을 줄이고 팀이 CDC 처리 로직과 Iceberg 적재 구조에 집중할 수 있도록 Managed Flink 를 선택함. (But—결국 내부구조를 알아야 한다는 것을 뼈저리게 학습) 이후 운영 경험이 쌓이면 Kubernetes 기반 배포로 확장할 수 있도록 애플리케이션 구조와 설정 주입 방식을 분리함.
- **Catalog—Glue:** Iceberg 테이블을 S3 warehouse 와 함께 운영해야 했기 때문에 AWS 환경과 자연스럽게 연결되는 Glue Catalog 를 사용함. 국내 빅테크 기업의 경우 자체 구현한 카탈로그를 사용하지만 도입 초기 단계에서 많이 사용되는 카탈로그로도 충분히 사용가능하다 판단함.

개발자 경험 주도 데이터플랫폼 모노레포 도입

2025.12 - 2026.02

Description: 데이터플랫폼의 파이프라인 코드와 공통 라이브러리를 단일 저장소(Monorepo)에서 관리할 수 있는 개발 환경을 구축했다. 패키지 관리 도구인 ‘uv’ 로 workspace 를 구성하고, 공통 라이브러리를 ‘symlink’ 로 연결해 프로젝트 간 의존성 중복을 줄였다. GitLab Runner 기반 CI/CD 를 함께 정리해 빌드, 테스트, 배포 흐름을 저장소 단위로 통합 관리했다.

• Problem

- 여러 Repository 에 파이프라인과 라이브러리가 분산되어 공통 의존성 관리가 파편화됨
- 프로젝트별 구조와 설정 방식이 달라 코드 일관성을 유지하기 어렵고 신규 Repository 셋업 비용이 증가함
- CI/CD 변경 시 Repository 별로 수정과 커뮤니케이션이 반복되어 개발 생산성이 저하됨

• Action

- ‘uv’ workspace 와 hatch build backend 를 활용해 프로젝트별 package/distribution 구조를 정리
- 공통 라이브러리를 ‘symlink’ 로 각 프로젝트에 연결해 중복 구현 없이 참조할 수 있도록 구성
- CI ‘changes’ 조건을 활용해 변경된 프로젝트만 선택적으로 빌드—테스트—배포하는 스크립트 구성
- 코드 품질 관리를 위해 ‘pre-commit(w/ pre)’ hook 을 도입하고 공통 검사 기준을 적용

• Result

- 공통 라이브러리의 수정, 테스트, 배포 흐름을 한 저장소에서 함께 다룰 수 있도록 정리
- 프로젝트마다 달랐던 개발 환경과 CI/CD 구성을 맞춰 신규 파이프라인 셋업 부담을 줄임
- 변경된 프로젝트만 빌드/테스트/배포하도록 구성해 Monorepo 전환 이후에도 CI/CD 실행 범위를 제어
- 공통 코드와 파이프라인 코드의 변경 영향도를 한곳에서 확인할 수 있어 리뷰와 배포 전 검토가 쉬워짐

• Impact

- 분산된 Python 프로젝트의 개발, 테스트, 배포 기준을 하나의 저장소 안에서 관리할 수 있게 됨
- 공통 라이브러리 재사용과 CI/CD 표준으로 개발자가 반복보다 비즈니스 로직에 집중할 수 있는 환경 마련
- 신규 파이프라인—신규 입사자가 기존 코드 구조, 공통 규칙, 배포 흐름을 더 빠르게 파악할 수 있는 기반 제공
- 데이터플랫폼 내 Python 프로젝트 운영 방식의 기준을 정리해 이후 파이프라인 확장과 유지보수 부담을 낮춤

• Self-Reflection

- 코드를 한 저장소로 모으는 작업을 넘어 구조, 규칙, 개발 흐름을 함께 정리하는 과정임을 체감
- 파이프라인의 의존성 관리 문제를 해결하면서 패키지 구조와 배포 단위를 함께 설계하는 경험을 쌓음
- uv, hatch, GitLab CI 를 조합—대규모 코드베이스에서 빌드 범위와 변경 영향도를 줄이는 방법을 학습
- Repository 분산 구조가 개발자 경험에 미치는 영향을 확인하고, 도구 선택만큼 운영 규칙과 팀 내 합의가 중요하다는 점을 배움

• Architecture Decision

- **uv**: Poetry 대비 의존성 해석과 설치 속도가 빠르고 workspace 구성이 단순해 Monorepo 환경의 기본 패키지 관리 도구로 선택함. 공통 라이브러리와 여러 파이프라인 프로젝트를 함께 다루는 구조에서 로컬 개발 경험과 CI 실행 시간을 줄이는 데 적합하다고 판단함. and Defacto.
- **hatch**: 오래된 build backend 조합보다 설정이 명확하고 현대적인 Python 패키징 흐름에 잘 맞아 도입함. Airflow 등 대형 오픈소스 프로젝트의 사용 사례를 참고했으며, 프로젝트별 distribution 관리와 향후 커스텀 빌드 확장에도 유리하다고 판단함.
- **prek**: 기존 pre-commit 대비 hook 실행 속도가 빠르고 사용성이 좋아 개발자가 로컬에서 품질 검사를 부담 없이 실행할 수 있다고 판단함. Airflow 사례를 통해 도구를 검토했고, uv 와 마찬가지로 빠른 피드백 루프를 만드는 방향에 맞아 선택함.

Data Quality Gate 개발—with Runtime/Semantic

2025.06 - 2025.08

Description: 파이프라인 전반의 데이터 품질을 관리하기 위해 Data Quality Gate 를 구축했다. Ingestion 단계의 'Runtime' 검증과 Transformation 단계의 'Semantic' 검증으로 책임을 나누고, Great Expectations 0.8.x 를 활용해 설정 파일 기반의 데이터 품질 검증 파이프라인을 구현했다.

• Problem

- 데이터 품질 이슈가 발생해도 어느 단계에서 문제가 생겼는지 파악하는 데 시간이 소요됨 (e.g—Cmd+Shift+f)
- BI 대시보드와 분석 결과의 신뢰도가 저하로 데이터 기반 의사결정에 부담이 생김
- 품질 검증을 수동으로 확인하는 과정이 반복되어 운영 리소스가 낭비됨

• Action

- Great Expectations(0.8.x) 를 도입해 데이터 품질 검증을 자동화할 수 있는 환경을 구성
- Runtime 검증: Ingestion 단계에서 데이터 형식, Null 여부 등 기본 정합성을 확인
- Semantic 검증: Transformation 단계에서 비즈니스 로직 기반의 통계 분포와 관계 무결성을 확인
- YAML 설정 파일로 검증 규칙(Expectation)을 정의하고 CLI 기반으로 파이프라인에 통합
- 품질 검증 실패 시 Airflow Task 실패 처리와 Slack 알림을 연동해 대응 흐름을 마련

• Result

- 품질 검증을 파이프라인 실행에 포함해 오류 데이터가 후속 단계로 넘어가기 전에 확인할 수 있도록 구성
- 품질 검증 리포트(Data Docs)를 자동 생성해 데이터 상태와 검증 결과를 함께 확인할 수 있게 됨
- Runtime/Semantic 검증 기준을 나누어 데이터 오류가 어느 단계에서 발생했는지 추적하기 쉬워짐
- 신규 데이터 파이프라인에도 같은 품질 검증 절차를 적용할 수 있도록 규칙 정의와 실행 방식을 정리

• Impact

- 파이프라인 실행 전후에 데이터 품질을 확인하는 절차가 생기며 분석 결과에 대한 신뢰도가 개선됨
- 데이터 분석가와 현업 사용자가 품질 이슈를 리포트 기준으로 확인할 수 있어 커뮤니케이션 비용이 줄어들음
- 품질 게이트를 통해 데이터 거버넌스 체계에서 필요한 검증, 기록, 알림의 기술적 기반을 마련
- 데이터 품질을 개별 담당자의 수동 확인이 아니라 파이프라인의 공통 책임으로 다루는 계기를 만들음

• Self-Reflection

- 데이터 품질은 특정 Task 에서 한 번 확인하는 것이 아니라 End-to-End 흐름 안에서 관리해야 함을 체감
- Great Expectations 를 활용해 검증 기준을 코드와 설정으로 관리하는 Expectations as Code 경험을 쌓음
- Runtime—Semantic으로 검증 책임을 나누면서 복잡한 파이프라인의 품질 기준을 더 명확히 설계할 수 있었음
- 도구 도입만으로는 품질 관리가 정착되지 않으며 운영 방식에 맞는 검증 프로세스가 함께 필요하다는 점을 배움

• Architecture Decision

- **Great Expectations:** 데이터 형식, Null, 통계 분포, 관계 무결성 등 다양한 검증 규칙을 Expectation 으로 표현할 수 있어 Runtime/Semantic 검증을 하나의 프레임워크 안에서 다루기 적합하다고 판단함. 다만 설정 파일과 실행 구조가 다소 무겁고, 단순 검증에도 프레임워크 의존성이 커지는 한계가 있었음. 당시에는 롤 확장성과 리포팅을 함께 제공하는 뚜렷한 오픈소스 대안을 찾기 어려워 Great Expectations 를 채택했지만, 이후 hash/checksum 비교나 chunk 단위 검증처럼 반복적이고 단순한 품질 검사는 별도 CLI 로 분리하는 방향을 검토중.

데이터 플랫폼 ReArchitect—Software Architecture Document 를 중심으로

2025.09 - 2025.11

Description: 데이터 플랫폼의 구성요소가 늘어나면서 시스템 구조, 운영 방식, 기술 선택의 이유를 일관되게 설명하기 어려운 상태였다. 이를 해결하기 위해 CMU SEI 의 Software Architecture Document(SAD) 템플릿을 참고해 데이터 플랫폼 아키텍처를 재정리하고, Architecture Decision Record(ADR)를 도입해 기술 선택과 운영 기준을 문서화했다. 문서는 Markdown 으로 작성하고, mkdocs 와 GitLab CI/CD, S3 static web hosting 을 통해 정적 문서 사이트로 제공했다.

• Problem

- 플랫폼 구성요소는 많아졌지만 문서가 산발적으로 흩어져 신규 엔지니어 온보딩에 시간이 소요됨
- Redshift 선택 이유나 운영 정책의 배경처럼 의사결정 맥락이 남지 않아 유사한 논의가 반복됨
- 운영 런북과 용어 기준이 부족해 팀 간 커뮤니케이션에서 해석 차이가 발생

• Action

- SEI SAD 템플릿을 기반으로 CnC/Deployment view 를 정리하고, Pipeline/Management 등 데이터 플랫폼 특성을 반영한 커스텀 view 를 추가
- Architecture Decision Record(ADR)를 도입해 기술 선택의 배경, 대안, 결정 기준, 변경 이력을 기록
- mkdocs 기반 정적 문서 빌드 파이프라인을 구성하고 GitLab CI/CD 와 S3 static web hosting 으로 문서 배포를 자동화
- 코드 변경 시 관련 문서 변경 여부를 확인할 수 있도록 CI 알림을 구성해 문서와 구현의 불일치를 줄임
- 온보딩 가이드와 운영 런북을 Markdown 으로 정리해 트러블슈팅과 플랫폼 운영 방식을 팀 내에 공유

• Result

- 신규 엔지니어가 플랫폼 구조, 주요 구성요소, 운영 절차를 문서 사이트에서 한 번에 확인할 수 있는 기반을 마련
- ADR 을 통해 기술 선택과 운영 정책의 배경을 남겨 반복적인 의사결정 논의를 줄임
- 구성요소, 파이프라인, 운영 런북을 같은 문서 체계 안에서 관리해 장애 대응과 변경 검토에 필요한 맥락을 찾기 쉬워짐
- 문서 빌드와 배포를 CI/CD 에 포함해 문서 변경 사항을 코드 리뷰 흐름 안에서 함께 관리할 수 있게 됨

• Impact

- 플랫폼을 코드 구현 중심이 아니라 설계 의도, 운영 기준, 품질 속성까지 포함한 아키텍처 관점에서 바라보게 됨
- 팀 간 책임 경계와 용어 기준을 문서로 맞추며 운영 혼선과 중복 논의를 줄일 수 있는 기반을 마련
- 기술 선택을 개인의 경험이나 관성에만 의존하지 않고 ADR 기반의 근거 있는 의사결정으로 남기는 문화를 만드는 데 기여

• Self-Reflection

- 기술 문서화는 단순한 기록이 아니라 조직이 시스템을 이해하고 설명하는 방식을 맞추는 작업임을 체감
- 코드, 운영, 의사결정의 관계를 구조적으로 관리하지 않으면 플랫폼 복잡도가 커질수록 혼란도 함께 커진다는 점을 배움
- 플랫폼 문서는 일관성을 유지하기 위한 운영 자산이며, 구현과 함께 지속적으로 갱신되어야 한다고 판단

- 이후에는 Quality Gate, Schema Evolution 등 운영 품질과 더 밀접한 영역까지 문서 체계를 확장할 필요성을 확인

DATA API

2025.07 - 2025.10

Description: 거래소 서비스에서 사용자별 최근 3년 거래금액 등 고비용 집계 데이터를 실시간에 가깝게 활용하고자 DW의 컴퓨팅 파워를 활용해 집계 데이터를 생성, Reverse ETL을 거쳐 RDB에 저장한 후 FastAPI 기반 API 서버로 제공했다. Airflow로 배치를 운영하고, Redshift에서 집계 후 MySQL로 적재하여 서비스팀에서 집계 -> 조회 가능한 형태의 API 레이어를 구축했다.

• Problem

- 고비용 집계 쿼리를 OLTP 에서 직접 수행할 경우 서비스 지연이나 장애로 이어질 위험이 있었음
- Redshift 를 서비스 API 의 직접 조회 대상으로 사용하기에는 보안, 동시성, 사용자 경험 측면의 부담이 컸음
- 서비스팀에서 집계 데이터를 활용하려면 DW 와 애플리케이션 사이를 연결하는 별도 서빙 레이어가 필요했음

• Action

- Airflow 기반 집계 파이프라인을 개발하고 서비스별 SLA 를 확인할 수 있는 모니터링 환경을 구성
- 무거운 집계는 Redshift 에서 수행하고, 조회에 필요한 결과만 MySQL 로 적재하는 Reverse ETL 구조를 설계
- FastAPI 기반 API 서버를 개발하고 Uvicorn worker 와 Gunicorn process manager 를 조합해 ASG 환경에서 운영
- 반복 조회 비용을 줄이기 위해 외부 캐시 대신 API 서버 내부의 로컬 캐시를 적용
- 여러 서비스 API 코드를 하나의 Repository 에서 관리하되, 서비스별 CD 파이프라인은 분리해 독립적으로 배포할 수 있도록 구성
- 이벤트 테스트와 데이터 생성 작업에 활용할 수 있는 인하우스 툴을 Django/Celery 기반으로 개발

• Result

- DW 에서 생성한 고비용 집계 결과를 MySQL 과 API 레이어를 통해 서비스에서 조회할 수 있는 구조를 마련
- 집계 연산과 서비스 조회 책임을 분리해 OLTP 부하를 늘리지 않고 집계 데이터를 제공할 수 있게 됨
- Airflow, Redshift, MySQL, FastAPI 로 이어지는 배치-서빙 흐름을 운영 가능한 형태로 연결
- 서비스별 API 코드를 Monorepo 에서 관리하면서도 배포 단위를 분리해 변경 영향 범위를 제어

• Impact

- DW 기반 집계 데이터를 서비스 API 로 제공하는 데이터 제품의 형태를 만들었음
- 데이터팀의 역할을 ETL 결과물 제공에서 서비스팀이 사용할 수 있는 API 레이어 설계와 운영까지 확장
- 무거운 연산은 DW 에 위임하고 서비스 트래픽 경로와 분리해 거래소 서비스의 운영 안정성을 높임

• Self-Reflection

- DW, 서빙 DB, API 로 이어지는 end-to-end 제품 구조를 설계하며 데이터 파이프라인과 애플리케이션 레이어를 연결하는 경험을 쌓음
- 서비스팀과 SLA/SLO 를 논의하면서 데이터 엔지니어링 결과물이 실제 사용자 경험과 운영 기준으로 이어지는 과정을 이해
- 배치 성공 여부뿐 아니라 API 응답, 캐시, 배포, 장애 대응까지 함께 고려해야 데이터 제품으로 운영할 수 있음을 배움

• Architecture Decision

- : TBU

Description: AWS RDS 의 변경 사항을 실시간으로 확보하기 위해 Debezium 기반 CDC(Change Data Capture) 파이프라인을 구축했다. Debezium Source Connector 가 RDS binlog 를 읽어 MSK 로 발행하고, Kafka Connect Sink Connector 를 통해 BigQuery 로 적재하는 구조였다. CDC 특성상 snapshot 부하, binlog retention, connector 재시작 시 offset 복구처럼 운영 난도가 높은 영역을 메트릭과 알람 기반으로 관리할 수 있도록 정비했다.

• Problem

- 분석팀과 서비스팀에서 변경 이력 기반 데이터가 필요했지만 기존 DW 파이프라인만으로는 삭제/갱신 이벤트를 포함한 history 를 얻기 어려웠음
- log_ 테이블은 비즈니스 로직과 스키마가 달라 실제 행 단위 변경 이벤트와 불일치하는 경우가 있었음
- CDC 는 지속 운영이 핵심인데 connector 지연, 에러, 적재 상태를 관측할 수 있는 기준이 부족했음

• Action

- DBA 와 협업해 writer DB 에 영향을 주지 않도록 snapshot.mode, locking.mode 등 snapshot/lock 운영 정책을 조정
- Kafka Connect 를 EKS 에 배포하고 Helm/ArgoCD 기반 GitOps 로 connector 설정을 Configuration as Code 형태로 관리
- AMP(Prometheus) 와 AMG(Grafana)를 연동해 JMX 지표 기반의 connector 상태, 이벤트 지연, 적재 흐름을 관측
- promotion 등으로 binlog position 이 변경되는 상황에 대비해 connector 복구 스크립트와 운영 문서를 작성

• Result

- RDS 변경 이벤트를 MSK 와 BigQuery 로 이어지는 CDC 파이프라인에서 안정적으로 수집할 수 있는 구조를 마련
- snapshot/lock 정책을 조정해 CDC 초기 적재 과정이 운영 DB 에 미치는 영향을 줄임
- connector 지연, 에러, 적재 상태를 Grafana 에서 확인할 수 있어 장애 징후를 더 빠르게 파악할 수 있게 됨
- binlog position 변경이나 connector 재시작 상황에 대응할 수 있는 복구 절차를 문서화

• Impact

- 기존 log 기반 파이프라인으로는 얻기 어려웠던 행 단위 변경 기록을 데이터 플랫폼에서 직접 수집할 수 있게 됨
- Kafka Connect 와 BigQuery 기반 CDC 흐름을 정리해 이후 여러 소스 시스템으로 확장 가능한 운영 패턴을 마련
- CDC 를 단순 적재가 아니라 관측성, 복구성, DB 영향도까지 함께 관리해야 하는 스트리밍 시스템으로 운영하는 경험을 쌓음

• Self-Reflection

- CDC 는 데이터를 옮기는 기능보다 offset, snapshot, lock, retention 을 안정적으로 관리하는 운영 역량이 중요하다는 점을 체감
- Debezium 과 Kafka Connect 는 문서만으로 이해하기 어려운 영역이 많아 장애와 성능 문제를 직접 다루며 내부 동작을 더 깊게 이해하게 됨
- 이후 플랫폼 도구를 도입할 때도 기능 목록보다 관측성, 복구성, 운영성을 먼저 확인하는 기준을 갖게 됨

• Architecture Decision

- **Debezium:** AWS DMS 와 비교했을 때 Kafka Connect 기반으로 운영할 수 있고, 로그 기반 CDC 에 필요한 snapshot, offset, schema change 경험이 오픈소스 생태계에 많이 축적되어 있다고 판단. 당시 Debezium 은 RDB 변경 로그를 Kafka 로 흘려보내는 CDC 영역에서 레퍼런스가 많고 여러 기업에서 검증한 선택지였음. Flink CDC 에 비해 설정과 운영은 다소 복잡하지만, 수집 계층을 Kafka 생태계 안에서 표준화하고 이후 sink 확장까지 고려하기에는 Debezium 이 더 적합하다고 봄. (당시 MSK Connect 는 개발 경험이 다소 부족했음)

Description: 기존 Kinesis + Lambda 기반 실시간 적재 파이프라인을 MSK(Kafka) + Kafka Connect(BigQuery Sink) 기반으로 전환했다. 거래소 회원 서비스의 EDA(Event-Driven Architecture) 전환에 맞춰 서버 로그와 이벤트를 안정적으로 수집/적재할 수 있는 표준 실시간 파이프라인을 구축했다. Connect cluster 는 Helm chart 로 구성하고 ArgoCD 기반 GitOps 로 배포해 확장성과 운영 일관성을 확보했다.

● Problem

- EDA 전환으로 실시간 이벤트 흐름을 Kafka 중심으로 일원화해야 했고, 기존 Kinesis 기반 파이프라인은 점진적으로 종료될 예정이었음
- Lambda 기반 BigQuery 적재는 스케일링과 오류 재처리 과정이 복잡했고 incident 상황에서 메시지 누락 여부를 확인하기 어려웠음
- 서버 로그 적재의 신뢰성을 확인할 수 있는 SLO 와 재처리 기준이 명확하지 않았음
- MSK 에서 BigQuery 로 적재하는 여러 방식 중 운영 복잡도, 성숙도, 확장성을 기준으로 표준 방식을 정해야 했음

● Action

- BigQuery Sink Connector 를 채택해 메시지 재처리, schema auto-create, source/sink 구성 분리를 지원하는 Kafka Connect 기반 구조로 전환
- Confluent 공식 Helm chart 를 기반으로 Connect cluster 를 커스터마이징하고 EKS/ArgoCD 기반 GitOps 배포 흐름을 구성
- connector task 상태, end-to-end 처리 지연, BigQuery 적재 상태를 Grafana 와 alert 로 모니터링할 수 있도록 구성
- connector 설정과 배포 방식을 표준화해 신규 이벤트 파이프라인을 같은 패턴으로 추가할 수 있도록 정리

● Result

- Kinesis/Lambda 기반 적재 흐름을 Kafka Connect 기반 표준 파이프라인으로 전환
- 신규 이벤트 파이프라인을 connector 설정 중심으로 추가할 수 있게 되어 반복 구현 부담이 줄어들음
- EDA 전환 서비스의 이벤트와 로그를 Kafka 중심으로 수집하면서 조직 내 실시간 데이터 흐름을 단순화
- self-hosted Connect cluster 운영 기준을 마련해 모니터링, 배포, 장애 대응을 일관된 방식으로 관리

● Impact

- Kafka Connect 를 통해 실시간 수집 파이프라인을 표준화하고 다양한 source/target 으로 확장 가능한 데이터 플랫폼 인터페이스를 마련
- Kinesis 에 분산되어 있던 실시간 적재 흐름을 Kafka 기반으로 정리해 유지보수 부담을 줄임
- 로그와 이벤트 수집의 신뢰성을 높여 이후 CDC, 실시간 분석, 피드백 모델링으로 확장할 수 있는 기반을 제공

● Self-Reflection

- Kafka Connect 는 구성만으로 끝나는 도구가 아니라 메트릭, 재처리, 버전 호환성, 스키마 일관성까지 함께 관리해야 함을 체감
- DLQ 전략, sink 재시도 정책, connector 재배포 타이밍 같은 운영 이슈를 겪으며 스트리밍 파이프라인의 핵심 관리 지점을 이해
- EKS, Helm, ArgoCD 기반 배포 자동화를 경험하며 데이터 플랫폼을 SRE/DevOps 관점에서 운영하는 역량을 넓힘

● Architecture Decision

- **Kafka Connect:** 당시 파이프라인의 책임은 복잡한 transform 보다 Kafka 토픽의 데이터를 BigQuery 로 안정적으로 옮기는 EL 영역에 가까웠음. Kafka Connect 는 Kafka native 하게 source/sink 를 연결하고 offset, 재처리, connector 운영 모델을 표준화할 수 있어 별도 애플리케이션을 직접 구현하는 것보다 적합하다고 판단. 복잡한 transform 은 dbt 등 downstream ELT 레이어에 두고, 수집 레이어는 Kafka 생태계의 connector 와 운영 도구를 활용하는 방향을 선택함.

Description: 기존 Bamboo + Airflow 기반 dbt 배포 파이프라인은 사용성이 낮고, stash 에서 Bamboo 와 Airflow 를 거쳐 결과를 확인해야 해 피드백 흐름이 길었다. GitHub Enterprise 도입을 계기로 CI/CD 체계를 GitHub Actions 기반으로 재설계했다. Slim CI, Full Build, Reviewdog, Jira 연동을 포함한 데이터팀 중심의 자동화 흐름을 구축해 dbt 개발과 배포의 일관성을 높였다.

● Problem

- Bamboo 기반 파이프라인은 빌드/테스트 결과를 확인하기까지 여러 단계를 거쳐야 해 개발자 경험이 좋지 않았음
- Airflow DAG 기반 배포 방식은 변경 이력과 실행 맥락이 분산되어 실패 원인을 추적하기 어려웠음
- 변경된 dbt 모델의 upstream/downstream 영향도를 충분히 확인하지 못한 채 전체 빌드를 반복하는 비효율이 있었음
- GitHub Enterprise 도입 이후 코드 리뷰와 검증, 배포 흐름을 한곳에서 관리할 수 있는 CI 체계가 필요했음

● Action

- GitHub Actions 기반 CI/CD 로 전환하고 dev/prod 브랜치별 workflow 를 분리
- `dbt build --select modified` 기반 Slim CI 와 Full Build 전략을 함께 설계
- Reviewdog, SQLFluff 를 도입해 PR 단계에서 dbt 모델과 SQL 품질을 자동 검증
- Jira 연동 action 을 활용해 PR 생성, 리뷰, merge 과정의 태깅과 이슈 트래킹을 자동화
- EKS 기반 self-hosted runner 를 설계해 빌드 환경을 표준화하고 실행 일관성을 확보
- S3 caching 전략을 적용해 반복 빌드에서 필요한 artifact 를 재사용할 수 있도록 구성

● Result

- GitHub Actions 안에서 dbt 빌드, 테스트, 린트, 배포 검증을 처리할 수 있는 CI/CD 흐름을 마련
- Slim CI 를 도입해 변경된 모델 중심으로 검증 범위를 좁히고 전체 빌드 반복을 줄임
- PR 단계에서 SQL 품질과 dbt 모델 변경 영향을 함께 확인할 수 있어 리뷰 흐름이 단순해짐
- self-hosted runner 와 caching 전략을 통해 빌드 환경과 실행 결과의 재현성을 높임

● Impact

- dbt 개발자가 코드 리뷰, 품질 검증, 배포 상태를 GitHub 중심으로 확인할 수 있게 되어 개발 경험이 개선됨
- 데이터팀의 공통 CI/CD workflow 를 마련해 신규 dbt 프로젝트에도 같은 검증 기준을 적용할 수 있게 됨
- lint 와 Reviewdog 기반 자동 검증으로 스키마와 쿼리 품질을 PR 단계에서 관리하는 문화가 정착됨

● Self-Reflection

- CI/CD 도구는 단순히 빌드를 실행하는 시스템이 아니라 개발 과정의 사용성, 속도, 가시성을 결정한다는 점을 체감
- dbt 의존성 그래프를 활용한 Slim CI 전략이 데이터 제품 개발자의 반복 검증 부담을 줄이는 데 효과적임을 배움
- GitHub Actions, EKS runner, lint, Jira 연동 경험을 이후 다른 데이터 파이프라인 자동화에도 확장할 수 있는 기반으로 삼음
- 재현성, 가시성, 자동화가 CI/CD 파이프라인의 핵심 운영 기준이라는 점을 명확히 이해하게 됨

Description: 사내 크루가 데이터 조직에 매번 요청하지 않고 데이터를 찾고, 적재하고, 변환하고, 탐색할 수 있는 Self-Serve Data 환경을 구축했다. DataHub(데이터 카탈로그), Airbyte(외부 데이터 파이프라인), MageAI(로우코드 데이터 워크플로우)를 Helm chart 와 ArgoCD 기반으로 배포했다. 데이터셀 의존도를 낮추고 내부 사용자가 필요한 데이터를 직접 탐색하고 활용할 수 있는 기반을 마련하는 것이 목표였다.

• Problem

- 내부 여러 조직이 필요한 데이터를 얻기 위해 매번 데이터셀에 요청하면서 데이터 활용 과정에 병목이 발생
- RDB, DW, 외부 API 등 데이터 위치를 파악하기 어렵고 메타데이터 관리가 부족해 재사용성이 낮았음
- 외부 데이터(거래소 API, CMC 등)를 개별 구현 방식으로 수집하면서 신뢰성과 유지보수에 부담이 있었음
- 비전문가도 사용할 수 있는 직관적인 self-serve 데이터 환경이 부족했음

• Action

- DataHub 구축:
 - Airflow 스크립트로 RDB information schema ingestion 을 자동화
 - dbt/BigQuery 는 recipe YAML 과 ingestion CLI 기반으로 메타데이터를 수집
 - Helm/ArgoCD 기반 배포와 RBAC, tag 정책을 구성
- Airbyte 도입:
 - Python CDK 기반 custom connector 를 개발하고 connector 저장소를 구성
 - semantic versioning 을 도입하고 GitHub Actions 에서 ECR 로 자동 빌드/배포
 - OAuth 기능 부재를 보완하기 위해 oauth2-proxy 를 연동
- MageAI 도입:
 - 외부 기관 보고용 정제 작업과 workflow 자동화를 구성
 - 공식 이미지의 PostgreSQL client 누락 문제를 확인하고 직접 PR 을 제출해 merge 경험 확보
 - Helm chart 를 수정해 shared-volume 기반 운영 구조를 구성
- 데이터 적재(EL), 탐색(Browse), 메타데이터(Document), 변환(Transform)을 연결하는 Self-Serve 구조를 정리

• Result

- 보안팀이 DataHub tag 기능으로 개인정보 레이블을 직접 관리할 수 있게 되어 데이터셀 요청을 줄임
- 외부 거래소 candle 데이터 수집을 Airbyte 기반으로 자동화해 수집 방식과 데이터 일관성을 정리
- MageAI 기반 workflow 로 외부 기관 보고용 정제 작업을 자동화하고 반복 수동 작업을 줄임
- 내부 사용자가 데이터 위치와 메타데이터를 직접 확인할 수 있는 self-serve 데이터 활용 기반을 마련

• Impact

- 데이터 조직의 역할을 요청 처리 중심에서 플랫폼 제공 중심으로 전환할 수 있는 기반을 마련
- DataHub, Airbyte, MageAI 등 OSS 를 Kubernetes 환경에서 운영하며 데이터 플랫폼 엔지니어링 역량을 강화
- 개별 조직이 필요한 데이터를 직접 찾고 활용할 수 있는 구조를 제공해 팀 간 데이터 활용 범위를 넓힘
- Helm/ArgoCD 기반 배포 방식을 표준화해 이후 신규 데이터 제품 도입 시 운영 일관성을 확보

• Self-Reflection

- Self-Serve 플랫폼은 도구를 설치하는 작업이 아니라 사용자의 데이터 workflow 를 쉽게 만드는 일임을 이해
- 성숙한 OSS 와 초기 단계 도구를 함께 운영하면서 각 도구의 내부 구조와 제약을 깊게 분석할 수 있었음
- OAuth, Helm chart 수정, custom connector 개발 같은 운영 이슈를 해결하며 플랫폼 운영 역량을 넓힘

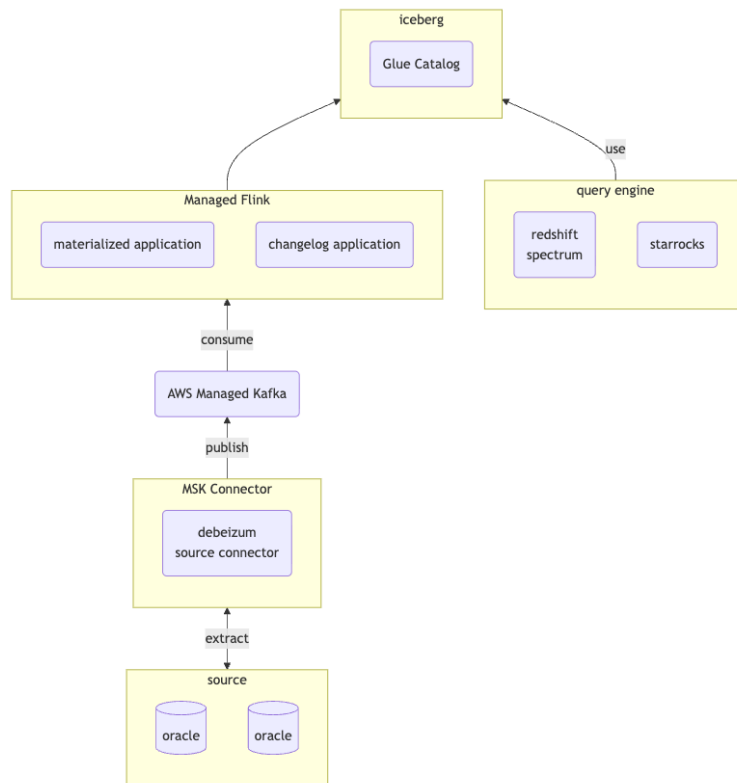


Figure 1: Near Realtime Pipeline

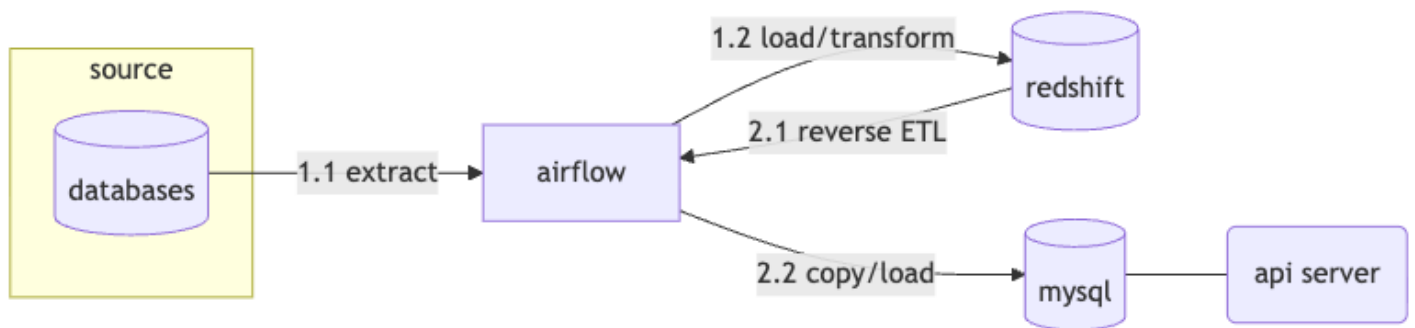


Figure 2: Near Realtime Pipeline-CICD

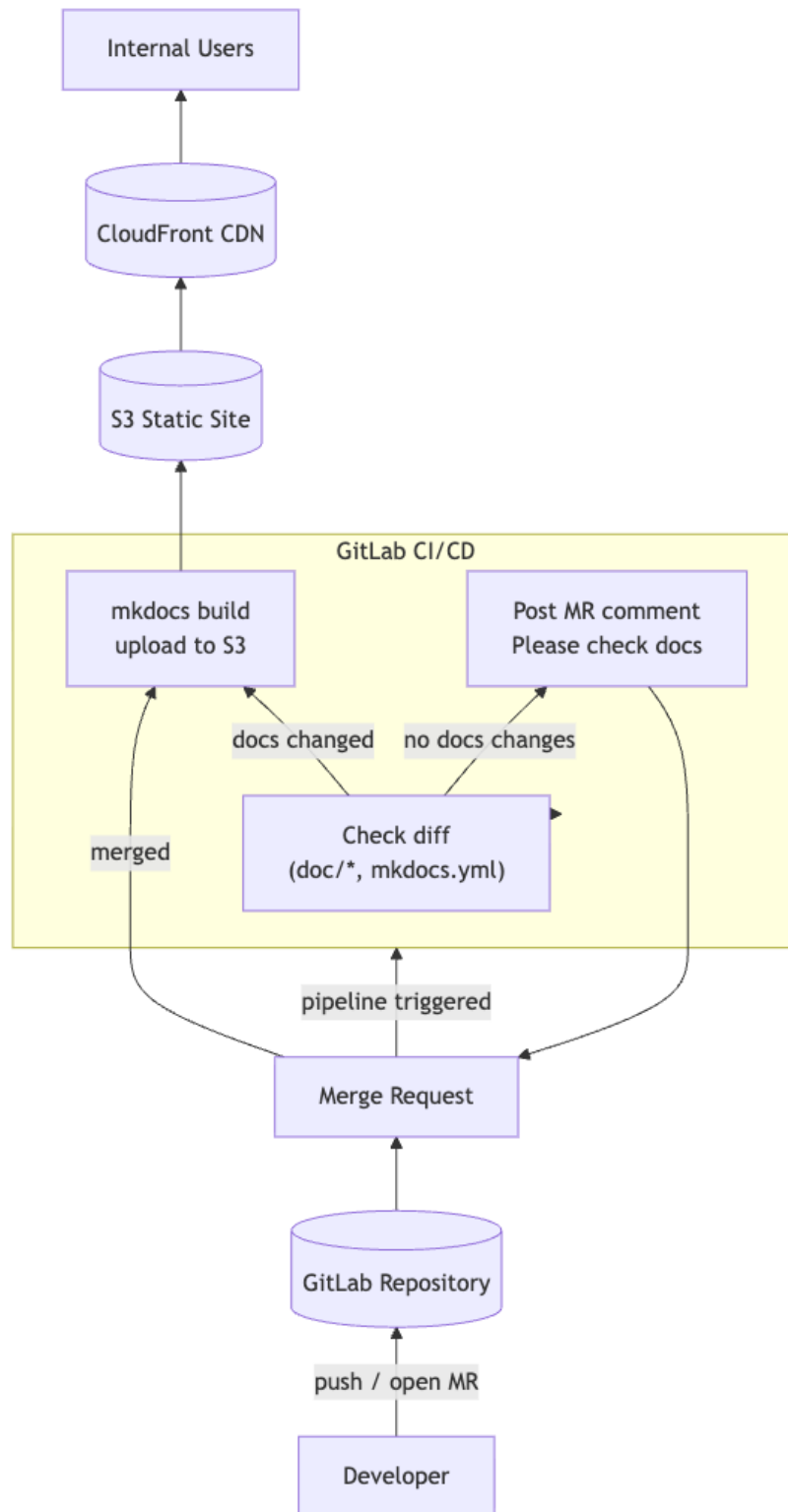


Figure 3: SAD CI/CD Pipeline Overview

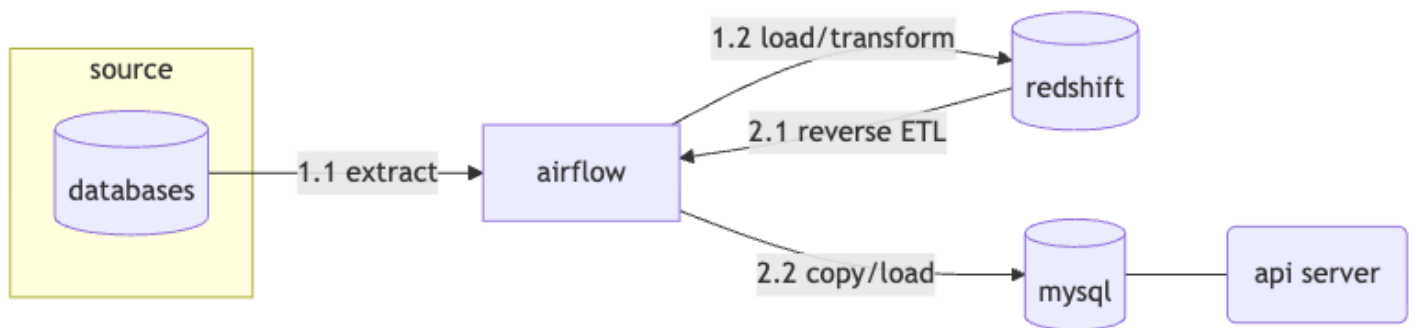


Figure 4: Data API Diagram

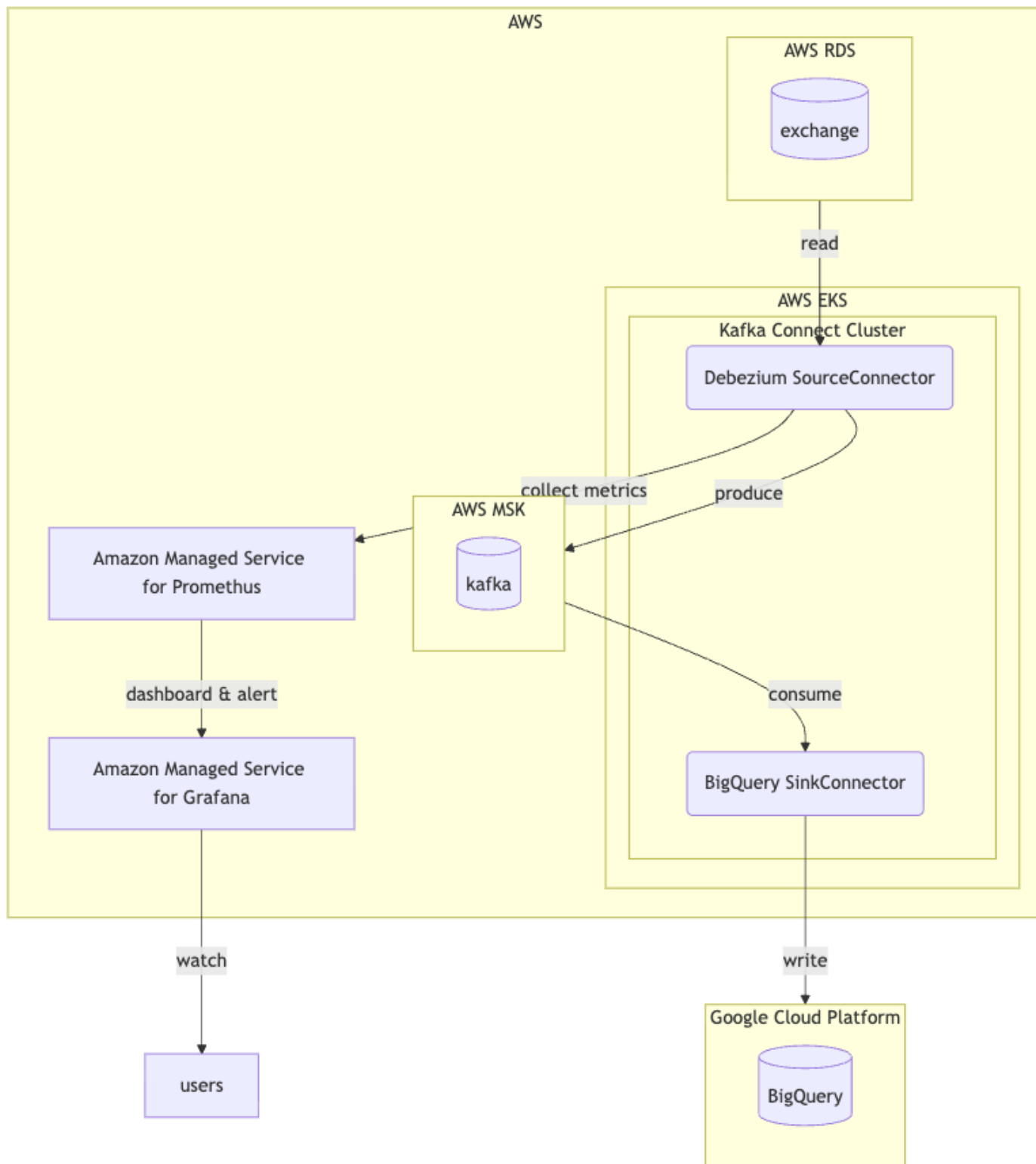


Figure 5: CDC Pipeline Overview

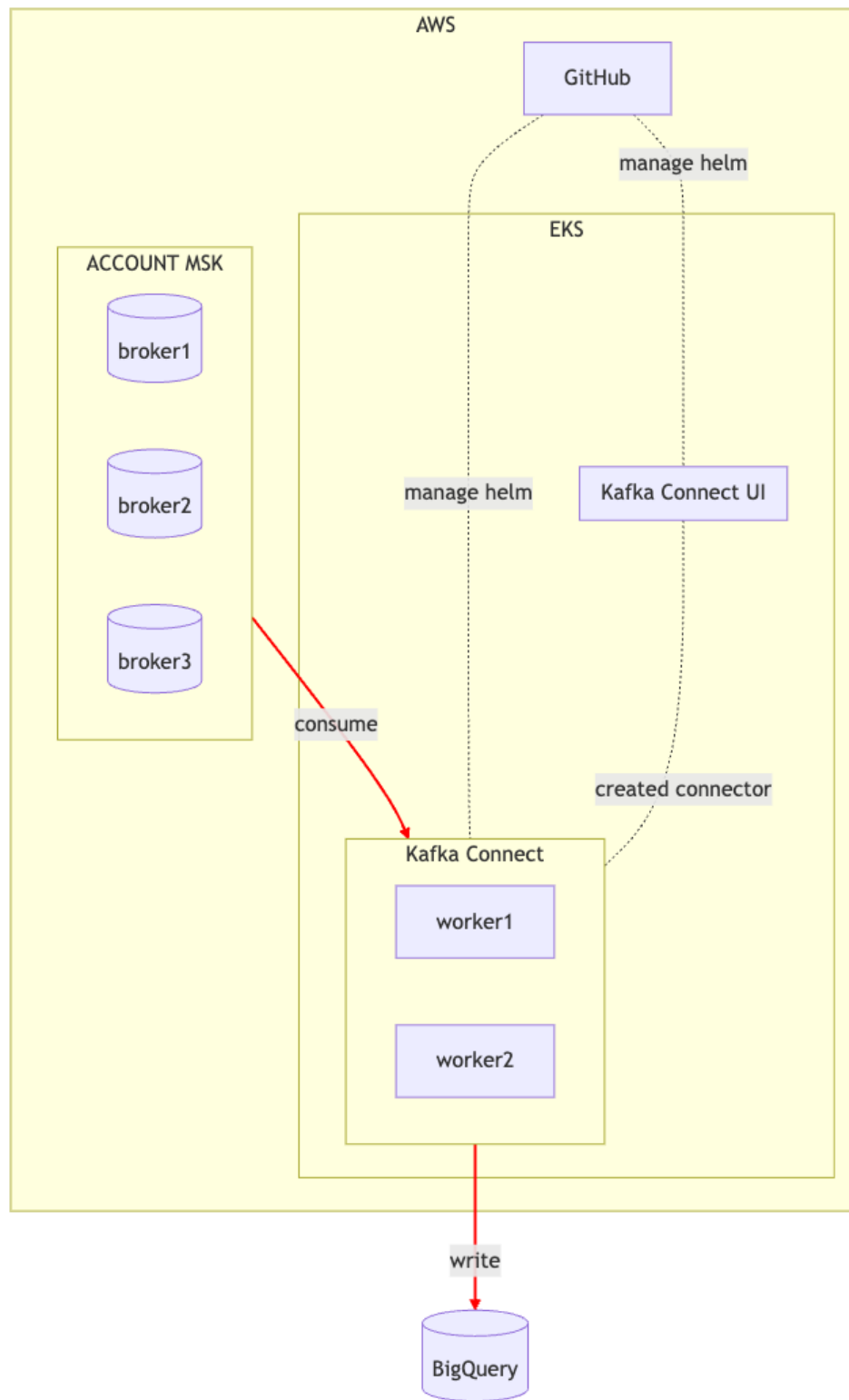


Figure 6: Kafka Connect Overview

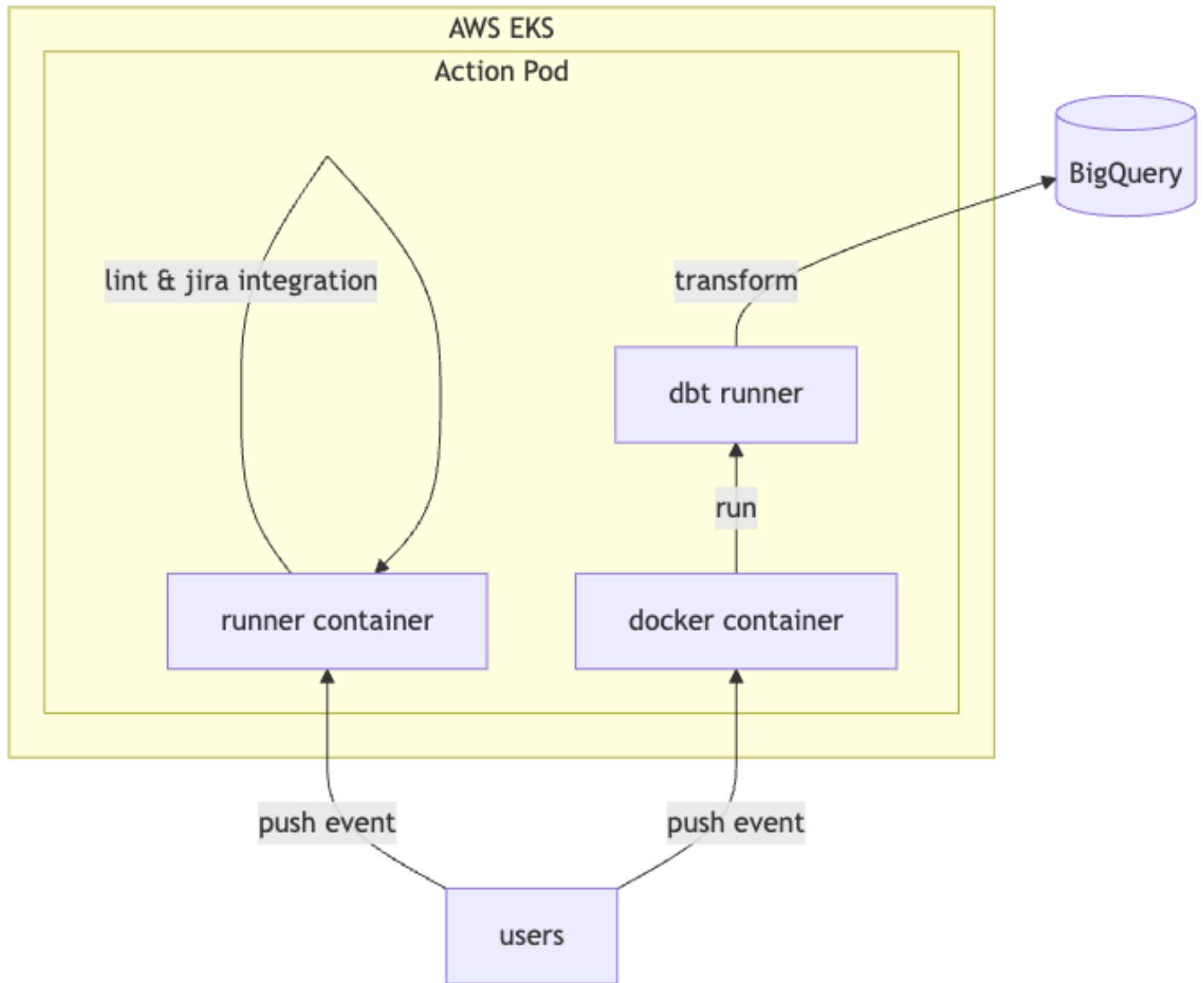


Figure 7: dbt CICD pipeline Overview